tapho-r workflow

User interface:

taphonomyR interface

```
library(TaphonomyR)


data("HomoninPartAbundance") # hpa
NewSite ← load.tapho.data("path/to/yourfile.xlsx")


plot.any.abundance.function(
  new.data = list(site1 = NewSite, site2 = HomoninPartAbundance),
  measure = c("MNI", "MNAU"),
  time.range = c("Upper Paleolithic", "Neolithic"),
  geological.context = "Cave",
  individual_number = c(5, 50),
  age = "Adults",
  gender = "M",
  cause = "cannibalism",
  bone_parts = c("Skull", "Arms"),
  bone.level = c(2, 1, 3, 4, 2, 1),
  site = c("Qafzeh", "Regourdou"),
  mode = "sum",
  hominin_species = c("Neanderthals", "Homo antecessor")
)
```

Dataloader:

load.tapho.data()

Purpose:
Processes Minimum Number of Elements (MNE) data to compute derived variables for taphonomic analysis.

Inputs:
Excel file containing MNE and "meta" data.

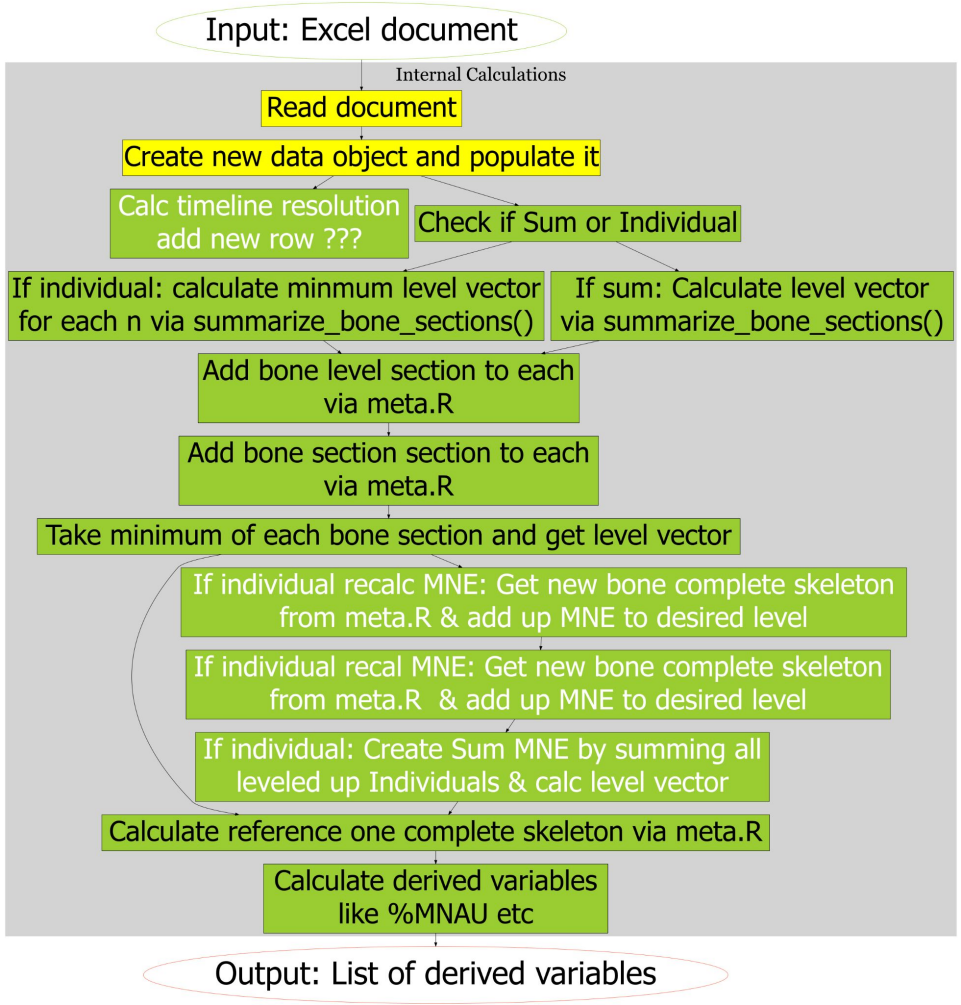
Outputs:
A list containing computed variables such as MNI, abundance, MAU, MNAU, percentages, and a summarized bone level vector.

list(MNE,Elements,abundance,MNI,MAU,MNAU,PercentMAU,PercentMNAU, ReMNAU,AcReMNAU,MeanAbundance,MeanPartAbundance,bone_level)

Error Handling:
Ensures consistency in MNE vector length relative to skeleton data and inconsistencies in level calculation; throws an error if mismatched.

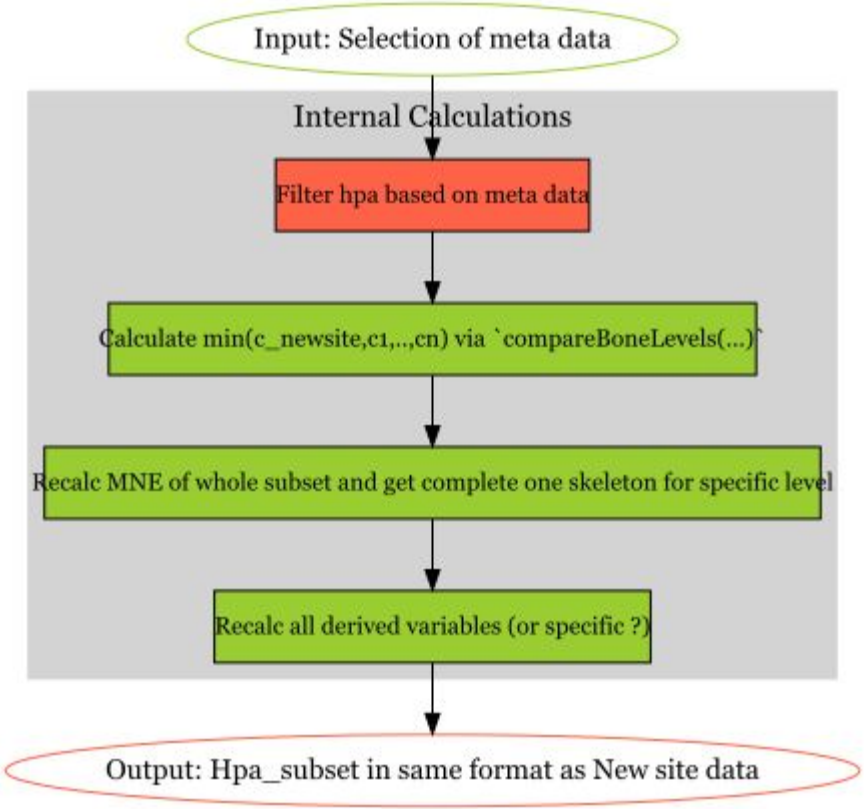
Dependencies: meta.R, summarize_bone_sections, convertMNEtoVariables, add_bone_info, get_oneskeleton

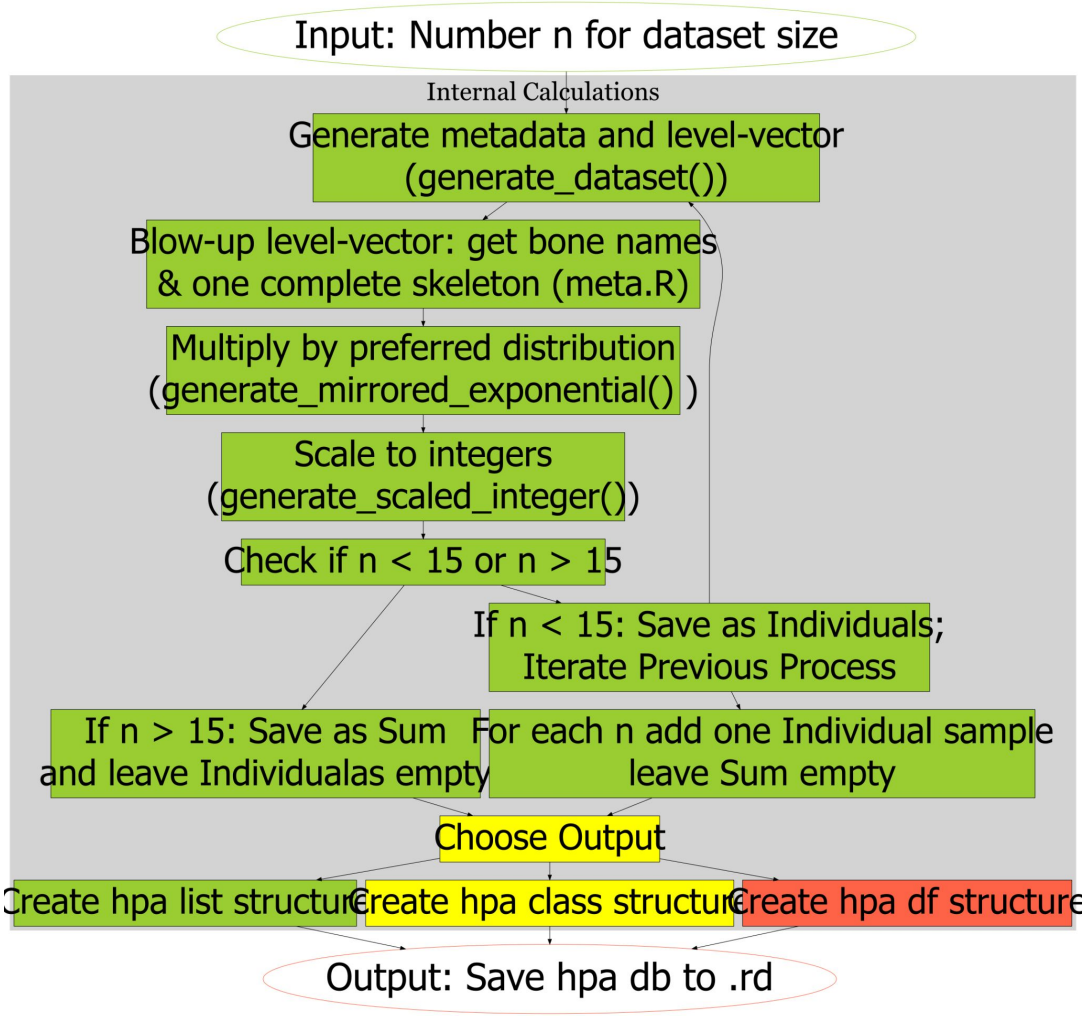
Dataloader:
load.tapho.data()



select_subset_hpa

```r
selected_data ← select_archaeological_data(
  data = HomoninPartAbundance,
  measure = c("MNI", "MNAU"),
  time.range = c("Upper Paleolithic", "Neolithic"),
  geological.context = "Cave",
  individual_number = c(5, 50),
  age = "Adults",
  gender = "M",
  cause = "cannibalism",
  bone_parts = c("Skull", "Arms"),
  bone.level = c(2, 1, 3, 4, 2, 1),
  site = c("Qafzeh", "Regourdou"),
  mode = "sum",
  hominin_species = c("Neanderthals", "Homo antecessor")
)
```




Input: Number n for dataset size
Internal Calculations
Generate metadata and level-vector (generate_dataset())
Blow-up level-vector: get bone names & one complete skeleton (meta.R)
Multiply by preferred distribution (generate_mirrored_exponential())
Scale to integers (generate_scaled_integer())
Check if n < 15 or n > 15
If n < 15: Save as Individuals; Iterate Previous Process
If n > 15: Save as Sum and leave Individualas empty
For each n add one Individual sample leave Sum empty
Choose Output
Create hpa list structure
Create hpa class structure
Create hpa df structure
Output: Save hpa db to .rd

class_structure

```
hpa_data = ArchaeologicalData(
  Site_1 = Site(
    sum = Sum(...),
    individuals = list(Individual(...), Individual(...), ...)
  ),
  Site_2 = Site(
    sum = NULL,
    individuals = list(Individual(...), ...)
  ),
  Site_3 = Site(
  sum = Sum(...),
  individuals = NULL
  ),
  ...
)
```

User interface:

```
library(TaphonomyR)

data("HomoninPartAbundance") # hpa
NewSite <- load.tapho.data("path/to/yourfile.xlsx")

plot.any.abundance.function(
  new.data = list(site1 = NewSite, site2 = HomoninPartAbundance),
  measure = c("MNI", "MNAU"),
  time.range = c("Upper Paleolithic", "Neolithic"),
  geological.context = "Cave",
  individual_number = c(5, 50),
  age = "Adults",
  gender = "M",
  cause = "cannibalism",
  bone_parts = c("Skull", "Arms"),
  bone.level = c(2, 1, 3, 4, 2, 1),
  site = c("Qafzeh", "Regourdou"),
  mode = "sum",
  hominin_species = c("Neanderthals", "Homo antecessor")
)
```

User interface:

```
library(TaphonomyR)

data("HomoninPartAbundance") # hpa
NewSite <- load.tapho.data("path/to/yourfile.xlsx")

plot.any.abundance.function(
  new.data = list(site1 = NewSite, site2 = HomoninPartAbundance),
  measure = c("MNI", "MNAU"),
  time.range = c("Upper Paleolithic", "Neolithic"),
  geological.context = "Cave",
  individual_number = c(5, 50),
  age = "Adults",
  gender = "M",
  cause = "cannibalism",
  bone_parts = c("Skull", "Arms"),
  bone.level = c(2, 1, 3, 4, 2, 1),
  site = c("Qafzeh", "Regourdou"),
  mode = "sum",
  hominin_species = c("Neanderthals", "Homo antecessor")
)
```

Dataloader:

step-by-step:

1. read.document turn it into dataframe

2. Calculate **c()-level** vector via *`summarize_bone_sections()`*,
    a. adds bone section column to bone
    b. adds bone level section to each bone
    c. taks minimum of each bone section

3. Calculates `One_skeleton` (reference complete skeleton)
    a. input c() -> meta.R look up-> ouput

4. Calculate derived variables *(%MNAU..)*

Main functions:

plot.any.abundance.function()

Purpose:
Creates plots comparing abundance measures of skeletal elements across different datasets, using user-specified parameters to filter and aggregate data.

Inputs:
Loaded tapho. data from `load.tapho.data()`.

Outputs:
Returns a ggplot/plotly object or a list of plots showing the specified measures across different datasets.

Key Features:
Data Integration, Data Filtering, Bone level Adjustment, Plot Generation

Dependencies:

Main functions:

step-by-step:

1. Select *HomoninPartAbundance (hpa)* **subset based on selection**,

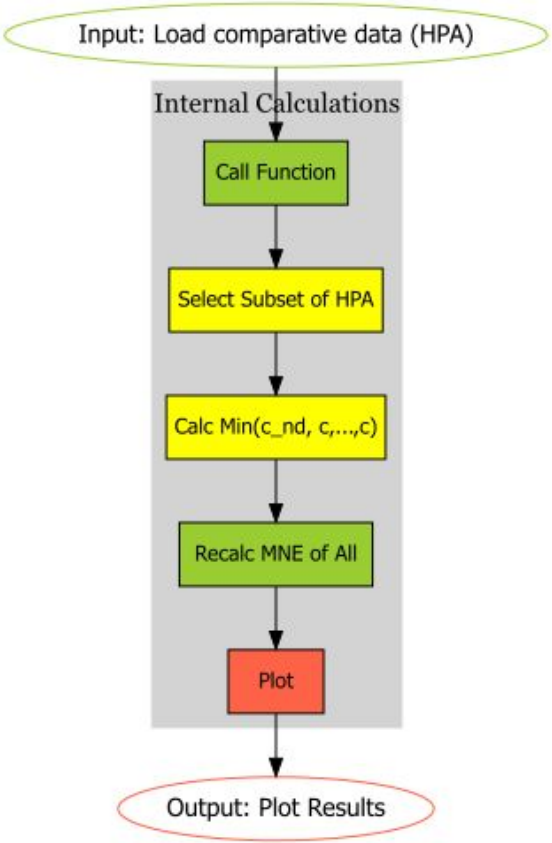
2. Calculate **min(c_newsite,c1,..,cn)** via *`compareBoneLevels(...)`*,

3. Recalculate MNE

4. Recalculate derived variables *(%MNAU)*

5. Select variables and feed plot.function

Workflow:


Input: Load comparative data (HPA)
Internal Calculations
Call Function
Select Subset of HPA
Calc Min(c_nd, c,...,c)
Recalc MNE of All
Plot
Output: Plot Results

Synthetic test data

Synthetic test data: Step I

generate_dataset(n_samples)

Purpose:
Generates a synthetic dataset containing archaeological and paleontological metadata.

Inputs:
A tibble with randomly sampled time ranges, geological contexts, individual counts, age categories, gender, accumulation causes, and hominin species.

Outputs:
A list containing computed variables such as MNI, abundance, MAU, MNAU, percentages, and a summarized bone level vector.

Error Handling:
Ensures consistency in MNE vector length relative to skeleton data and inconsistencies in level calculation; throws an error if mismatched.

Dependencies:

Synthetic test data: Step II

load.tapho.data()

Purpose:
Processes Minimum Number of Elements (MNE) data to compute derived variables for taphonomic analysis.

Inputs:
Excel file containing MNE and "meta" data.

Outputs:
A list containing computed variables such as MNI, abundance, MAU, MNAU, percentages, and a summarized bone level vector.

Error Handling:
Ensures consistency in MNE vector length relative to skeleton data and inconsistencies in level calculation; throws an error if mismatched.

Dependencies:

Synthetic test data: userinterface

load.tapho.data()

Purpose:
Processes Minimum Number of Elements (MNE) data to compute derived variables for taphonomic analysis.

Inputs:
Excel file containing MNE and "meta" data.

Outputs:
A list containing computed variables such as MNI, abundance, MAU, MNAU, percentages, and a summarized bone level vector.

Error Handling:
Ensures consistency in MNE vector length relative to skeleton data and inconsistencies in level calculation; throws an error if mismatched.

Dependencies:

helper functions

helper functions:

plot.any.abundance.function()

Purpose:
Creates plots comparing abundance measures of skeletal elements across different datasets, using user-specified parameters to filter and aggregate data.

Inputs:
Loaded tapho. data from `load.tapho.data()`.

Outputs:
Returns a ggplot/plotly object or a list of plots showing the specified measures across different datasets.

Key Features:
Data Integration, Data Filtering, Bone level Adjustment, Plot Generation

Dependencies:

helper functions:

plot.any.abundance.function()

Purpose:
Creates plots comparing abundance measures of skeletal elements across different datasets, using user-specified parameters to filter and aggregate data.

Inputs:
Loaded tapho. data from `load.tapho.data()`.

Outputs:
Returns a ggplot/plotly object or a list of plots showing the specified measures across different datasets.

Key Features:
Data Integration, Data Filtering, Bone level Adjustment, Plot Generation

Dependencies:

helper functions:

plot.any.abundance.function()

Purpose:
Creates plots comparing abundance measures of skeletal elements across different datasets, using user-specified parameters to filter and aggregate data.

Inputs:
Loaded tapho. data from `load.tapho.data()`.

Outputs:
Returns a ggplot/plotly object or a list of plots showing the specified measures across different datasets.

Key Features:
Data Integration, Data Filtering, Bone level Adjustment, Plot Generation

Dependencies:

helper functions:

plot.any.abundance.function()

Purpose:
Creates plots comparing abundance measures of skeletal elements across different datasets, using user-specified parameters to filter and aggregate data.

Inputs:
Loaded tapho. data from `load.tapho.data()`.

Outputs:
Returns a ggplot/plotly object or a list of plots showing the specified measures across different datasets.

Key Features:
Data Integration, Data Filtering, Bone level Adjustment, Plot Generation

Dependencies:

helper functions:

plot.any.abundance.function()

Purpose:
Creates plots comparing abundance measures of skeletal elements across different datasets, using user-specified parameters to filter and aggregate data.

Inputs:
Loaded tapho. data from `load.tapho.data()`.

Outputs:
Returns a ggplot/plotly object or a list of plots showing the specified measures across different datasets.

Key Features:
Data Integration, Data Filtering, Bone level Adjustment, Plot Generation

Dependencies:

helper functions:

plot.any.abundance.function()

Purpose:
Creates plots comparing abundance measures of skeletal elements across different datasets, using user-specified parameters to filter and aggregate data.

Inputs:
Loaded tapho. data from `load.tapho.data()`.

Outputs:
Returns a ggplot/plotly object or a list of plots showing the specified measures across different datasets.

Key Features:
Data Integration, Data Filtering, Bone level Adjustment, Plot Generation

Dependencies: